Control Flow

Decisions and Loops in Python

LEARNING OUTCOME

By the end of this module you will write all conditional structures, all loop types with break/continue/else, use list comprehensions, and understand Python-specific iteration patterns.

Skills You Will Gain
• Write if/elif/else decision structures
• Use match/case (Python 3.10+) pattern matching
• Write for and while loops correctly
• Use break, continue, and loop else clause
• Create list, dict, and set comprehensions
• Use enumerate, zip, and range effectively

■ Control flow is what makes programs intelligent -- they make decisions and repeat actions. Python's clean syntax makes even complex logic readable and Pythonic.

SECTION 1

Conditionals

```
# if / elif / else score = 85 if score >= 90: grade = "A" elif score >= 80: grade = "B" elif score >= 70: grade = "C" else: grade = "F" print(f"Grade: {grade}") # Ternary (conditional expression) status = "adult" if age >= 18 else "minor" result = x if x > 0 else -x # abs value # Truthy / Falsy values # Falsy: False, 0, 0.0, "", [], {}, (), set(), None # Truthy: everything else if users: # True if list not empty print("Has users") if name: # True if string not empty print(f"Hi {name}") if data is not None: # explicit None check process(data) # Python 3.10+ match/case (structural pattern matching) command = "quit" match command: case "quit" | "exit" | "q": print("Goodbye!") case "help" | "h": show_help() case "go" if direction: move(direction) case _: # default (wildcard) print(f"Unknown: {command}")
```

SECTION 2

Loops

```
# for loop -- iterate over any iterable for i in range(5): # 0,1,2,3,4 print(i) for i in range(2, 10, 2): # 2,4,6,8 (start,stop,step) print(i) for i in range(10, 0, -1): # countdown 10..1 print(i) # Iterate lists, strings, dicts fruits = ["apple","banana","cherry"] for fruit in fruits: print(fruit) for ch in "Python": print(ch) person = {"name":"Alice","age":28,"city":"Mumbai"} for key, value in person.items(): print(f"{key}: {value}") # enumerate -- index + value for i, fruit in enumerate(fruits): print(f"{i}: {fruit}") # 0: apple, 1: banana... for i, fruit in enumerate(fruits, start=1): # start from 1 print(f"{i}. {fruit}") # zip -- iterate multiple iterables together names = ["Alice","Bob","Carol"] scores = [95, 87, 92] for name, score in zip(names, scores): print(f"{name}: {score}") # zip stops at shortest -- use zip_longest for all from itertools import zip_longest for a, b in zip_longest([1,2,3],[10,20],fillvalue=0): print(a, b) # while loop count = 0 while count < 5: print(count) count += 1 # while with break (read until "quit") while True: cmd = input("Enter command: ") if cmd == "quit": break print(f"Running: {cmd}") # break, continue, else for n in range(10): if n == 3: continue # skip 3 if n == 7: break # stop at 7 print(n) # 0 1 2 4 5 6 else: print("Loop completed without break") # only if no break!
```

SECTION 3

Comprehensions

```
# LIST COMPREHENSION -- [expression for item in iterable if condition] squares = [x**2 for x in range(10)] # [0,1,4,9,16,25,36,49,64,81] evens = [x for x in range(20) if x%2==0] # [0,2,4,6,8,10...] uppercased = [s.upper() for s in ["a","b","c"]] # ["A","B","C"] flattened = [n for row in matrix for n in row] # flatten 2D list # Nested comprehension pairs = [(x,y) for x in range(3) for y in range(3) if x!=y] # DICT COMPREHENSION square_map = {x: x**2 for x in range(5)} # {0:0, 1:1, 2:4...} word_len = {w: len(w) for w in ["hi","hello","hey"]} swapped = {v: k for k,v in original.items()} # swap keys/values # SET COMPREHENSION unique_lens = {len(w) for w in words} # unique word lengths # GENERATOR EXPRESSION -- like list comprehension but lazy (no list created) gen = (x**2 for x in range(1000000)) # no memory allocated yet total = sum(x**2 for x in range(1000)) # efficient any_even = any(x%2==0 for x in nums) # short-circuits all_pos = all(x>0 for x in nums) # When to use what: # List comp: when you need the whole list in memory # Generator expr: when you only iterate once (sum, any, all, max) # Dict comp: when building a dict from transformation # Set comp: when you need unique values # Walrus operator := (Python 3.8+) -- assign AND use in expression while chunk := file.read(8192): process(chunk) if (n := len(data)) > 10: print(f"Too long: {n} items")
```

■ PRACTICE Q & A

Q1. What is the else clause on a loop?

A: *The else block runs only if the loop completed WITHOUT a break statement. Useful for search loops: if you find what you're looking for and break, else doesn't run. If you exhaust the loop without finding it, else runs.*

Q2. What is the difference between a list comprehension and a generator expression?

A: *List comp [x for x in range(n)] creates the entire list in memory immediately. Generator (x for x in range(n)) is lazy -- yields one item at a time. Use generators for large datasets or when iterating only once.*

Q3. What is enumerate() and why use it instead of range(len())?

A: *enumerate() gives index + value pairs while iterating. Pythonic and readable. Avoid for i in range(len(lst)): items[i] -- use for i,item in enumerate(lst): instead.*

Q4. What does the walrus operator := do?

A: *Assigns a value to a variable and also returns that value in the same expression. Useful for avoiding double calls: while line := file.readline(). Python 3.8+.*

Q5. What are the falsy values in Python?

A: *False, 0, 0.0, 0j, empty string '', empty list [], empty dict {}, empty tuple (), empty set set(), and None. Everything else is truthy.*

■ Control Flow Cheat Sheet	
`if x: / elif: / else:`	Conditional blocks
`x if cond else y`	Ternary expression
`for item in iterable:`	Iterate any iterable
`for i,v in enumerate(lst):`	Index and value
`for a,b in zip(lst1,lst2):`	Parallel iteration
`range(start,stop,step)`	Generate number range
`while condition:`	Loop while true
`break / continue`	Exit loop / skip iteration
`for..else:`	Else runs if no break
`[x*2 for x in lst if x>0]`	List comprehension
`{k:v for k,v in d.items()}`	Dict comprehension
`(x*2 for x in lst)`	Generator expression (lazy)